

David总板子_2

主要包含数论、字符串、计算几何、动态规划、杂项等模板

David总板子_2

数论

排列组合

数论杂项

线性代数

字符串

字典树

AC自动机

马拉车求最长回文串

KMP

字符串哈希

字符串常用库函数

计算几何

2D

3D

动态规划

斜率优化DP

杂项

高精度加减乘除

对拍

bf.cpp

solve.cpp

data.cpp

test.cpp

dpL.sh

dpW.bat

STL库函数

迭代器操作

数论

排列组合

```
1 //模逆元 排列数
2 vector<int> fac(maxn + 5), inv(maxn + 5);
3 int ksm(int x, int a, int mod){
4     int ans = 1;
5     while(a){
6         if(a & 1) ans = ans * x % mod;
7         x = x * x % mod;
8         a >>= 1;
9     }
10    return ans;
11 }
12
13 void init(){
14     fac[0] = 1;
```

```

15     for(int i = 1; i <= maxn; i++){
16         fac[i] = fac[i - 1] * i % mod;
17     }
18     inv[maxn] = ksm(fac[maxn], mod - 2, mod);
19     for(int i = maxn - 1; i >= 0; i--){
20         inv[i] = inv[i + 1] * (i + 1) % mod;
21     }
22 }
23
24 int A(int n, int m){
25     return fac[n] * inv[m] % mod;
26 }
27
28 int C(int n, int m){
29     return fac[n] * inv[m] % mod * inv[n - m] % mod;
30 }
31
32 //错位排列Derangement
33 int Derangement(int n){
34     vector<int> dp(n + 1);
35     dp[1] = 0;
36     dp[2] = 1;
37     for(int i = 3; i <= n; i++){
38         dp[i] = (n - 1) * (dp[i - 1] + dp[i - 2]) % mod;
39     }
40 }
41
42 //卡特兰数CatalanNumber 解决不交叉问题的种类数
43 //有 n 个左括号与 n 个右括号。能组成多少个合法括号序列?
44 int CatalanNumber(int n){
45     int ans = C(2 * n, n) - C(2 * n, n + 1);
46     return (ans % mod + mod) % mod;
47 }
48

```

数论杂项

```

1 //欧拉筛
2 const int mod = 1e9 + 7;
3 const int maxn = 1e6;
4 vector<int> primes;
5 //欧拉筛 同时 预处理最小质因数 与 目标数分解质因数
6
7 int min_primes[maxn + 5];
8 void init(){
9     for(int i = 2; i <= maxn; i++){
10         if(!min_primes[i]){
11             min_primes[i] = i;
12             primes.push_back(i);
13         }
14         for(int p : primes){
15             if(p > min_primes[i] || i * p > maxn) break;
16             min_primes[i * p] = p;
17         }
18     }
19 }
20 //分解质因数
21 vector<int> getPrimes(int x){

```

```

22     vector<int> res;
23     while(x > 1){
24         int p = min_primes[x];
25         res.push_back(p);
26         while(x % p == 0) x /= p;
27     }
28     return res;
29 }
30 //分解质因数带次数(幂) {p, cnt}
31 vector<pair<int,int>> getPrimesCnt(int x){
32     vector<pair<int,int>> res;
33     while(x > 1){
34         int p = min_primes[x];
35         int cnt = 0;
36         while(x % p == 0){
37             x /= p;
38             cnt++;
39         }
40         res.push_back({p, cnt});
41     }
42     return res;
43 }
44
45
46 // 旧欧拉筛单独函数
47 // void init(){
48 //     vector<bool> nop(maxn + 5);
49 //     nop[0] = nop[1] = true;
50 //     for(int i = 2; i <= maxn; i++){
51 //         if(!nop[i]){
52 //             primes.push_back(i);
53 //             if(i > sqrt(maxn)) continue;
54 //             for(int j = i * i; j <= maxn; j++){
55 //                 nop[j] = true;
56 //             }
57 //         }
58 //     }
59 // }
60
61
62 int ksm(int x, int a, int m){
63     int ans = 1;
64     while(a){
65         if(a & 1) ans = ans * x % m;
66         x = x * x % m;
67         a >>= 1;
68     }
69     return ans;
70 }
71
72
73 //扩展欧几里得算法
74 int exgcd(int a, int b, int &x, int &y){
75     if(!b){
76         x = 1;
77         y = 0;
78     }
79     int d = exgcd(b, a % b, x, y);
80     int t = x;
81     x = y;
82     y = t - (a / b) * y;

```

```

83     return d;//最大公约数
84 }
85
86 //中国剩余定理( $x \equiv a \pmod r$ )
87 int CRT(int k, vector<int> a, vector<int> r){
88     int n = 1, ans = 0;
89     for(int i = 0; i < k; i++){
90         n = n * r[i];
91     }
92     for(int i = 0; i < k; i++){
93         int m = n / r[i], b, y;
94         exgcd(m, r[i], b, y);
95         ans = (ans + a[i] * m * b % n) % n;
96     }
97     return (ans % n + n) % n;
98 }
99
100 //扩展中国剩余定理
101 //求解方程组  $x \equiv a[i] \pmod{m[i]}$ 
102 int excrt(vector<int> m, vector<int> a, int n){
103     int M = m[0];
104     int ans = a[0];
105     for(int i = 1; i < n; i++){
106         int c = (a[i] - ans % m[i] + m[i]) % m[i];
107         int x, y;
108         int d = exgcd(M, m[i], x, y);
109         if(c % d != 1) return -1;
110         int k = m[i] / d;
111         x = ksm(x, c / d, k);
112         ans += x * M;
113         M *= k;
114         ans = (ans % M + M) % M;
115     }
116     return ans;
117 }
118
119 //一个数的欧拉函数
120 int oula_phi(int n){
121     int ans = n;
122     for(int i = 2; i * i <= n; i++){
123         if(n % i == 0){
124             ans = ans / i * (i - 1);
125             while(n % i == 0) n /= i;
126         }
127         if(n > 1) ans = ans / n * (n - 1);
128     }
129     return ans;
130 }
131
132 //筛法求欧拉函数
133 vector<int> sieve_phi(int n) {
134     vector<int> phi(n + 1);
135     vector<bool> st(n + 1, false);
136     vector<int> primes;
137
138     phi[1] = 1; // 特殊情况: 1的欧拉函数值为1[1,3,5](@ref)
139
140     for (int i = 2; i <= n; i++) {
141         if (!st[i]) { // i是质数
142             primes.push_back(i);
143             phi[i] = i - 1; // 质数的欧拉函数值为i-1[1,2](@ref)

```

```

144     }
145
146     for (int j = 0; primes[j] <= n / i; j++) {
147         st[primes[j] * i] = true;
148
149         if (i % primes[j] == 0) {
150             // primes[j]是i的最小质因子
151             phi[primes[j] * i] = phi[i] * primes[j]; // 情况1[2,5](@ref)
152             break;
153         } else {
154             // primes[j]与i互质
155             phi[primes[j] * i] = phi[i] * (primes[j] - 1); // 情况2[1,3](@ref)
156         }
157     }
158 }
159
160 return phi;
161 }
162
163 // 狄利克雷卷积快速幂 - 计算f的k次卷积幂
164 void dirichlet_power(const vector<int>& f, vector<int>& res, int n, int k) {
165     // 初始化结果为狄利克雷卷积的单位元ε
166     res.assign(n + 1, 0);
167     res[1] = 1; // ε(1)=1, ε(n)=0(n>1)
168
169     vector<int> temp(f), base(f);
170
171     while (k) {
172         if (k & 1) {
173             vector<int> new_res(n + 1, 0);
174             // 计算res = res * base
175             for (int i = 1; i <= n; i++) {
176                 for (int j = 1; i * j <= n; j++) {
177                     new_res[i * j] = (new_res[i * j] + res[i] * base[j]) % mod;
178                 }
179             }
180             res = new_res;
181         }
182
183         // 计算base = base * base
184         vector<int> new_base(n + 1, 0);
185         for (int i = 1; i <= n; i++) {
186             for (int j = 1; i * j <= n; j++) {
187                 new_base[i * j] = (new_base[i * j] + base[i] * base[j]) % mod;
188             }
189         }
190         base = new_base;
191
192         k >>= 1;
193     }
194 }
195
196 // 阶乘表, 通常n不会太大, 可以预先计算好
197 // 实际使用时可以根据需要动态计算或传入
198 const int fact[10] = {1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880};
199
200 /**
201  * 康托展开, 求排列的字典序排第几, O(n^2)
202  * @param permutation 排列, 元素为0..n-1或1..n均可, 但需是连续不重复的整数
203  * @return 康托展开值 (即比当前排列小的排列个数)
204  */

```

```

205 int cantor_expand(const std::vector<int>& permutation) {
206     int n = permutation.size();
207     int code = 0;
208
209     for (int i = 0; i < n; ++i) {
210         int smaller = 0; // 统计未出现的元素中有多少个比当前元素小
211         for (int j = i + 1; j < n; ++j) {
212             if (permutation[j] < permutation[i]) {
213                 smaller++;
214             }
215         }
216         code += smaller * fact[n - i - 1];
217     }
218
219     return code;
220 }
221
222 /**
223  * 逆康托展开
224  * @param code 康托展开值（比目标排列小的排列个数）
225  * @param n 排列长度
226  * @param elements 可选的元素集合，默认使用0..n-1；如果是1..n则需传入对应集合
227  * @return 对应的排列
228  */
229 std::vector<int> cantor_expand_inv(int code, int n, const std::vector<int>&
elements = {}) {
230     std::vector<int> result;
231
232     // 确定可用的元素集合
233     std::vector<int> available;
234     if (elements.empty()) {
235         // 默认使用0..n-1
236         for (int i = 0; i < n; ++i) {
237             available.push_back(i);
238         }
239     } else {
240         available = elements;
241     }
242
243     for (int i = n; i > 0; --i) {
244         int fact_val = fact[i - 1];
245         int index = code / fact_val; // 确定应该取第几小的数
246         code %= fact_val; // 更新余数
247
248         result.push_back(available[index]);
249         available.erase(available.begin() + index);
250     }
251
252     return result;
253 }
254
255
256 // FFT 需要用 double，所以局部不建议用全局的 int int 覆盖所有变量
257 struct Complex {
258     double r, i;
259     Complex(double r = 0, double i = 0) : r(r), i(i) {}
260     Complex operator + (const Complex& t) const { return {r + t.r, i + t.i}; }
261     Complex operator - (const Complex& t) const { return {r - t.r, i - t.i}; }
262     Complex operator * (const Complex& t) const { return {r * t.r - i * t.i, r *
t.i + i * t.r}; }
263 };

```

```

264
265 const double PI = acos(-1.0);
266 vector<int> rev; // 预处理位逆序
267
268 void fft_init(int len) {
269     rev.assign(len, 0);
270     for (int i = 0; i < len; i++)
271         rev[i] = (rev[i >> 1] >> 1) | ((i & 1) ? (len >> 1) : 0);
272 }
273
274 void fft(vector<Complex>& a, int type) {
275     int n = a.size();
276     for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
277     for (int mid = 1; mid < n; mid <= 1) {
278         Complex wn(cos(PI / mid), type * sin(PI / mid));
279         for (int i = 0; i < n; i += (mid << 1)) {
280             Complex w(1, 0);
281             for (int j = 0; j < mid; j++, w = w * wn) {
282                 Complex x = a[i + j], y = w * a[i + j + mid];
283                 a[i + j] = x + y;
284                 a[i + j + mid] = x - y;
285             }
286         }
287     }
288     if (type == -1) for (auto& x : a) x.r /= n;
289 }
290
291 // 多项式乘法封装
292 vector<int> multiply_fft(vector<int>& A, vector<int>& B) {
293     int n = A.size(), m = B.size(), total = n + m - 1;
294     int len = 1; while (len < total) len <= 1;
295     fft_init(len);
296     vector<Complex> fa(len), fb(len);
297     for (int i = 0; i < n; i++) fa[i].r = A[i];
298     for (int i = 0; i < m; i++) fb[i].r = B[i];
299     fft(fa, 1); fft(fb, 1);
300     for (int i = 0; i < len; i++) fa[i] = fa[i] * fb[i];
301     fft(fa, -1);
302     vector<int> res(total);
303     for (int i = 0; i < total; i++) res[i] = (int)(fa[i].r + 0.5);
304     return res;
305 }
306
307 // // 假设输入两个多项式的系数
308 //     vector<int> a = {1, 2, 1}; // x^2 + 2x + 1
309 //     vector<int> b = {1, 1};    // x + 1
310
311 //     int n = a.size(), m = b.size();
312 //     int len = 1;
313 //     while (len < n + m - 1) len <= 1; // 补齐到2的幂次
314
315 //     vector<Complex> fa(len), fb(len);
316 //     for (int i = 0; i < n; i++) fa[i] = Complex(a[i], 0);
317 //     for (int i = 0; i < m; i++) fb[i] = Complex(b[i], 0);
318
319 //     fft(fa, len, 1);
320 //     fft(fb, len, 1);
321
322 //     // 点值相乘
323 //     for (int i = 0; i < len; i++) fa[i] = fa[i] * fb[i];
324

```

```

325 //      fft(fa, len, -1);
326
327 //      // 输出结果 (四舍五入)
328 //      for (int i = 0; i < n + m - 1; i++) {
329 //          cout << (int)(fa[i].r + 0.5) << " ";
330 //      }
331 //      return 0;
332
333
334 const int NTT_MOD = 998244353;
335 const int G = 3; // 原根
336 const int GI = 332748118; // 原根的逆元
337
338 // 借用你原本的 ksm, 但注意模数要用 NTT_MOD
339 int ntt_ksm(int a, int b) {
340     int res = 1; a %= NTT_MOD;
341     while (b) {
342         if (b & 1) res = res * a % NTT_MOD;
343         a = a * a % NTT_MOD;
344         b >>= 1;
345     }
346     return res;
347 }
348
349 void ntt(vector<int>& a, int type) {
350     int n = a.size();
351     for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
352     for (int mid = 1; mid < n; mid <= 1) {
353         int wn = ntt_ksm(type == 1 ? G : GI, (NTT_MOD - 1) / (mid <= 1));
354         for (int i = 0; i < n; i += (mid <= 1)) {
355             int w = 1;
356             for (int j = 0; j < mid; j++, w = w * wn % NTT_MOD) {
357                 int x = a[i + j], y = w * a[i + j + mid] % NTT_MOD;
358                 a[i + j] = (x + y) % NTT_MOD;
359                 a[i + j + mid] = (x - y + NTT_MOD) % NTT_MOD;
360             }
361         }
362     }
363     if (type == -1) {
364         int inv_n = ntt_ksm(n, NTT_MOD - 2);
365         for (int& x : a) x = x * inv_n % NTT_MOD;
366     }
367 }
368
369 // NTT 多项式乘法封装
370 vector<int> multiply_ntt(vector<int> A, vector<int> B) {
371     int n = A.size(), m = B.size(), total = n + m - 1;
372     int len = 1; while (len < total) len <= 1;
373     fft_init(len); // rev数组计算逻辑与FFT一致
374     A.resize(len); B.resize(len);
375     ntt(A, 1); ntt(B, 1);
376     for (int i = 0; i < len; i++) A[i] = A[i] * B[i] % NTT_MOD;
377     ntt(A, -1);
378     A.resize(total);
379     return A;
380 }
381
382 // // 多项式 A: 1 + 2x + x^2
383 //     vector<int> A = {1, 2, 1};
384 //     // 多项式 B: 1 + x
385 //     vector<int> B = {1, 1};

```



```

386
387 //      // 直接调用封装好的乘法函数
388 //      vector<int> C = multiply_ntt(A, B);
389 //      vector<int> D = multiply_fft(A, B);
390
391 //      for (int x : C) {
392 //          cout << x << " ";
393 //      }
394 //      cout << endl;
395 //      // 输出预期: 1 3 3 1
396
397
398 // 类欧几里得算法 log求和 sum = i(0 - n-1) floor [(a*i + b) / c]
399 // 求i从0到n-1的ai+b/c向下取整的总和
400 // n >= 0, m > 0, a >= 0, b >= 0
401 int floor_sum(int n, int m, int a, int b) {
402     int ans = 0;
403     if (a >= m) {
404         ans += (n - 1) * n / 2 * (a / m);
405         a %= m;
406     }
407     if (b >= m) {
408         ans += n * (b / m);
409         b %= m;
410     }
411
412     int y_max = (a * n + b) / m;
413     int x_max = (y_max * m - b);
414     if (y_max == 0) return ans;
415
416     ans += (n - (x_max + a - 1) / a) * y_max;
417     ans += floor_sum(y_max, a, m, (a - x_max % a) % a);
418     return ans;
419 }
420
421 //SOSDP
422 int main() {
423     int n = 50;
424     int total_states = (1 << n);
425     vector<int> dp(total_states, 0); // dp数组初始化, 根据题意可能初始为A[i]
426
427     // SOS DP 计算超集和
428     for (int i = 0; i < n; ++i) {
429         for (int j = 0; j < total_states; ++j) {
430             if ((j >> i) & 1) { // 如果状态j的第i位是1
431                 dp[j] += dp[j ^ (1 << i)]; // 累加不含第i位的状态的值
432             }
433         }
434     }
435
436     // 此时dp[mask]存储了所有满足 (i & mask) == mask 的A[i]之和
437     return 0;
438 }

```

线性代数

```

1 const int mod = 1e9 + 7;
2

```

```

3 //线性基LinearBasis
4 vector<int> p(64);
5 bool insert(int x){
6     for(int i = 63; i >= 0; i--){
7         if(!(x >> i)) continue;
8         if(!p[i]){
9             p[i] = x;
10            return true;
11        }
12        x ^= p[i];
13    }
14    return false;
15 }
16
17 //矩阵 (0-based)
18 struct mat{
19     int n, m;
20     vector<vector<int>> a;
21     mat(int x = 0, int y = 0){
22         n = x;
23         m = y;
24         a.resize(x, vector<int> (y));
25     }
26
27     mat operator+ (const mat& b){
28         mat c(n, m);
29         for(int i = 0; i < n; i++){
30             for(int j = 0; j < m; j++){
31                 c.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
32             }
33         }
34         return c;
35     }
36
37     mat operator- (const mat& b){
38         mat c(n, m);
39         for(int i = 0; i < n; i++){
40             for(int j = 0; j < m; j++){
41                 c.a[i][j] = ((a[i][j] - b.a[i][j]) % mod + mod) % mod;
42             }
43         }
44         return c;
45     }
46
47     mat operator* (const mat& b){
48         mat c(n, b.m);
49         for(int i = 0; i < n; i++){
50             for(int j = 0; j < b.m; j++){
51                 for(int k = 0; k < m; k++){
52                     c.a[i][j] = (c.a[i][j] + a[i][k] * b.a[k][j]) % mod;
53                 }
54             }
55         }
56         return c;
57     }
58
59     mat transpose(){
60         mat res(m, n);
61         for(int i = 0; i < n; i++){
62             for(int j = 0; j < m; j++){
63                 res.a[j][i] = a[i][j];

```

```

64         }
65     }
66     return res;
67 }
68
69 void print(){
70     for(int i = 0; i < n; i++){
71         for(int j = 0; j < m; j++){
72             cout << a[i][j] << " ";
73         }
74         cout << '\n';
75     }
76 }
77
78 };
79
80 mat ksm(mat x, int a){
81     int n = x.a.size();
82     mat res(n, n);
83     for(int i = 0; i < n; i++){
84         res.a[i][i] = 1;
85     }
86     while(a){
87         if(a & 1) res = res * x;
88         x = x * x;
89         a >>= 1;
90     }
91     return res;
92 }
93

```

字符串

字典树

```

1  const int N=3e6+10;
2  const int base=13331;
3  const int mod=1e9+7;
4  int n,_T,q;
5  string s;
6  int ch[N][70]={0};
7  int idx;
8  int cnt[N];
9
10 int hashn(char x)
11 {
12     if(x>='A'&&x<='Z')
13     {
14         return x-'A';
15     }
16     else if(x>='a'&&x<='z')
17     {
18         return x-'a'+26;
19     }
20     else
21         return x-'0'+52;
22 }

```

```

23
24 void insert(string s)
25 {
26     int p=0;
27     int len=s.length();
28     for(int i=0;i<len;i++)
29     {
30         int j=hashn(s[i]);
31         if(!ch[p][j]) ch[p][j]=++idx;
32         p=ch[p][j];
33         cnt[p]++;
34     }
35
36 }
37
38 int query(string s)
39 {
40     int p=0;
41     int len=s.length();
42     for(int i=0;i<len;i++)
43     {
44         int j=hashn(s[i]);
45         if(!ch[p][j]) return 0;
46         p=ch[p][j];
47     }
48     return cnt[p];
49 }
50
51 void solve()
52 {
53     idx=0;
54     cin>>n>>q;
55     for(int i=1;i<=n;i++)
56     {
57         cin>>s;
58         insert(s);
59     }
60     while(q--)
61     {
62         cin>>s;
63         cout<<query(s)<<endl;
64     }
65     for(int i=0;i<=idx;i++)
66     {
67         for(int j=0;j<=65;j++)
68         {
69             ch[i][j]=0;
70         }
71     }
72     for(int i=0;i<=idx;i++)
73     {
74         cnt[i]=0;
75     }
76 }
77

```

AC自动机

```

1  const int N=(1e4+10)*55;
2  const int M=1e6+10;
3  int n,k,_n,T,idx;
4  char s[M];
5  int cnt[N];
6  int ne[N];
7  int ch[N][26];
8
9  void insert()
10 {
11     int p=0;
12     for(int i=0;s[i];i++)
13     {
14         int j=s[i]-'a';
15         if(!ch[p][j]) ch[p][j]=++idx;
16         p=ch[p][j];
17     }
18     cnt[p]++;
19 }
20
21 void build()
22 {
23     queue<int> q;
24     for(int i=0;i<26;i++)
25     {
26         if(ch[0][i]) q.push(ch[0][i]);
27     }
28     while(!q.empty())
29     {
30         int u=q.front();
31         q.pop();
32         for(int i=0;i<26;i++)
33         {
34             int v=ch[u][i];
35             if(v) ne[v]=ch[ne[u]][i],q.push(v);
36             else ch[u][i]=ch[ne[u]][i];
37         }
38     }
39 }
40
41 int query()
42 {
43     int ans=0;
44     for(int k=0,i=0;s[k];k++)
45     {
46         i=ch[i][s[k]-'a'];
47         for(int j=i;j&&~cnt[j];j=ne[j])
48         {
49             ans+=cnt[j];
50             cnt[j]=-1;
51         }
52     }
53     return ans;
54 }
55
56
57 void solve()
58 {
59     idx=0;
60     memset(cnt,0,sizeof cnt);
61     memset(ne,0,sizeof ne);

```

```

62     memset(ch,0,sizeof ch);
63     cin>>n;
64     for(int i=1;i<=n;i++)
65     {
66         cin>>s;
67         insert();
68     }
69     build();
70     cin>>s;
71     cout<<query()<<endl;
72 }
73

```

马拉车求最长回文串

```

1  {
2      d[1]=1;
3      for(int i=2,l=0,r=1;i<=n;i++)
4      {
5          if(i<=r) d[i]=min(r-i+1,d[r-i+1]);
6          while(s[i-d[i]]==s[i+d[i]]) d[i]++;
7          if(i+d[i]-1>r) l=i-d[i]+1,r=i+d[i]-1;
8      }
9  }
10

```

KMP

```

1  const int N=1e6+10;
2  int n,m;
3  char s1[N],s2[N];
4  string s,p;
5  int ne[N];
6
7  signed main(){
8      ios::sync_with_stdio(false);
9      cin.tie(0);
10     cout.tie(0);
11
12     cin>>s>>p;
13     int m=s.length(),n=p.length();
14     ne[1]=0;
15     for(int i=1;i<=m;i++)
16     {
17         s1[i]=s[i-1];
18     }
19     for(int i=1;i<=n;i++)
20     {
21         s2[i]=p[i-1];
22     }
23     for(int i=2,j=0;i<=n;i++)
24     {
25         while(j&& s2[i]!=s2[j+1]) j=ne[j];
26         if(s2[i]==s2[j+1]) j++;
27         ne[i]=j;
28     }
29 }

```

```

28     }
29     for(int i=1,j=0;i<=m;i++)
30     {
31         while(j&&s1[i]!=s2[j+1]) j=ne[j];
32         if(s1[i]==s2[j+1]) j++;
33         if(j==n)
34         {
35             cout<<i-n+1<<endl;
36             j=ne[j];
37         }
38     }
39
40     for(int i=1;i<=n;i++)
41     {
42         cout<<ne[i]<<' ';
43     }
44
45     return 0;
46 }

```

字符串哈希

```

1 //字符串哈希
2 struct StringHash {
3     static const int MAXN = 1e6 + 5;
4     static const int BASE = 277;
5     static const int MOD1 = 1e9 + 7;
6     static const int MOD2 = 1e9 + 9;
7
8     inline static vector<int> p1, p2;
9     vector<int> h1, h2;
10    int n;
11
12    // 初始化静态成员数组 p1 和 p2
13    static void get_power() {
14        static bool initialized = false;
15        if (!initialized) {
16            p1.assign(MAXN, 0);
17            p2.assign(MAXN, 0);
18            p1[0] = 1;
19            p2[0] = 1;
20            for (int i = 1; i < MAXN; i++) {
21                p1[i] = p1[i - 1] * BASE % MOD1;
22                p2[i] = p2[i - 1] * BASE % MOD2;
23            }
24            initialized = true;
25        }
26    }
27
28    // 构造函数：传入字符串，自动计算前缀哈希
29    StringHash(const string& s) {
30        get_power();
31        n = s.size();
32        h1.assign(n + 1, 0);
33        h2.assign(n + 1, 0);
34        for (int i = 0; i < n; i++) {
35            h1[i + 1] = (h1[i] * BASE + s[i]) % MOD1;
36            h2[i + 1] = (h2[i] * BASE + s[i]) % MOD2;

```

```

37     }
38 }
39
40 // 获取子串 [l, r] 的双哈希值 (0-indexed)
41 // 例如 s = "abcde", get(1, 3) 返回 "bcd" 的哈希
42 pair<int, int> get(int l, int r) {
43     int len = r - l + 1;
44     int res1 = (h1[r + 1] - h1[l] * p1[len] % MOD1 + MOD1) % MOD1;
45     int res2 = (h2[r + 1] - h2[l] * p2[len] % MOD2 + MOD2) % MOD2;
46     return {res1, res2};
47 }
48 };
49 /*
50 使用示例
51 string s1;
52 string s2;
53 StringHash hash1(s1);
54 StringHash hash2(s2);
55
56 pair<int, int> get1=hash1.get(1,2);
57 pair<int, int> get2=hash2.get(5,6);
58
59 //判断字符串s1的[1,2]与字符串s2的[5,6]是否完全匹配
60 if(get1.first==get2.first&&get1.second==get2.second){
61     cout<<"Yes\n";
62 }else{
63     cout<<"No\n";
64 }
65 */
66
67 struct MatrixHash {
68     static const int MAXN = 1005; // 最大行数
69     static const int MAXM = 1005; // 最大列数
70     static const int BASE_R = 13331; // 行基数
71     static const int BASE_C = 277; // 列基数
72     static const int MOD1 = 1e9 + 7;
73     static const int MOD2 = 1e9 + 9;
74
75     // 静态成员数组: 行幂次与列幂次
76     inline static vector<int> pr1, pr2, pc1, pc2;
77
78     // 二维前缀哈希表
79     vector<vector<int>> h1, h2;
80     int R, C;
81
82     // 初始化静态幂次数组
83     static void get_power() {
84         static bool initialized = false;
85         if (!initialized) {
86             pr1.assign(MAXN, 0); pr2.assign(MAXN, 0);
87             pc1.assign(MAXM, 0); pc2.assign(MAXM, 0);
88             pr1[0] = pr2[0] = pc1[0] = pc2[0] = 1;
89
90             for (int i = 1; i < MAXN; i++) {
91                 pr1[i] = 1LL * pr1[i - 1] * BASE_R % MOD1;
92                 pr2[i] = 1LL * pr2[i - 1] * BASE_R % MOD2;
93             }
94             for (int i = 1; i < MAXM; i++) {
95                 pc1[i] = 1LL * pc1[i - 1] * BASE_C % MOD1;
96                 pc2[i] = 1LL * pc2[i - 1] * BASE_C % MOD2;
97             }
98         }
99     }
100 };

```



```

98         initialized = true;
99     }
100 }
101
102 // 注意传入数组是 字符串 还是 int !!!!
103 // 构造函数: 传入 vector<string> 或 vector<vector<int>>
104 MatrixHash(const vector<string>& mat) {
105     get_power();
106     R = mat.size();
107     C = mat[0].size();
108     h1.assign(R + 1, vector<int>(C + 1, 0));
109     h2.assign(R + 1, vector<int>(C + 1, 0));
110
111     for (int i = 0; i < R; i++) {
112         for (int j = 0; j < C; j++) {
113             // 二维哈希递推公式:  $H[i][j] = H[i-1][j]*Br + H[i][j-1]*Bc - H[i-1][j-1]*Br*Bc + val$ 
114             h1[i + 1][j + 1] = (1LL * h1[i][j + 1] * BASE_R % MOD1 +
115                                1LL * h1[i + 1][j] * BASE_C % MOD1 -
116                                1LL * h1[i][j] * BASE_R % MOD1 * BASE_C %
MOD1 +
117                                mat[i][j] + MOD1) % MOD1;
118
119             h2[i + 1][j + 1] = (1LL * h2[i][j + 1] * BASE_R % MOD2 +
120                                1LL * h2[i + 1][j] * BASE_C % MOD2 -
121                                1LL * h2[i][j] * BASE_R % MOD2 * BASE_C %
MOD2 +
122                                mat[i][j] + MOD2) % MOD2;
123         }
124     }
125 }
126
127 // 获取子矩阵 [(r1, c1), (r2, c2)] 的双哈希值 (0-indexed)!!!
128 pair<int, int> get(int r1, int c1, int r2, int c2) {
129     int dr = r2 - r1 + 1; // 行高
130     int dc = c2 - c1 + 1; // 列宽
131
132     // 提取公式:  $H = h[r2][c2] - h[r1-1][c2]*Br^{dr} - h[r2][c1-1]*Bc^{dc} + h[r1-1][c1-1]*Br^{dr}*Bc^{dc}$ 
133     int res1 = (h1[r2 + 1][c2 + 1] - 1LL * h1[r1][c2 + 1] * pr1[dr] % MOD1 +
MOD1) % MOD1;
134     res1 = (res1 - 1LL * h1[r2 + 1][c1] * pc1[dc] % MOD1 + MOD1) % MOD1;
135     res1 = (res1 + 1LL * h1[r1][c1] * pr1[dr] % MOD1 * pc1[dc] % MOD1) %
MOD1;
136
137     int res2 = (h2[r2 + 1][c2 + 1] - 1LL * h2[r1][c2 + 1] * pr2[dr] % MOD2 +
MOD2) % MOD2;
138     res2 = (res2 - 1LL * h2[r2 + 1][c1] * pc2[dc] % MOD2 + MOD2) % MOD2;
139     res2 = (res2 + 1LL * h2[r1][c1] * pr2[dr] % MOD2 * pc2[dc] % MOD2) %
MOD2;
140
141     return {res1, res2};
142 }
143 };
144

```

```

1  #include <cstring>
2
3  char str1[20] = "hello";
4  char str2[20] = "world";
5
6  // 字符串长度
7  size_t len = strlen(str1);          // 5
8
9  // 字符串拷贝
10 strcpy(str1, "new");                // str1 = "new"
11 strncpy(str1, "hello", 3);          // 安全拷贝
12
13 // 字符串连接
14 strcat(str1, " world");              // str1 = "hello world"
15 strncat(str1, "!!!", 2);            // 安全连接
16
17 // 字符串比较
18 int cmp = strcmp(str1, str2);        // 返回0相等, <0小于, >0大于
19 int cmp_n = strncmp(str1, str2, 3);  // 比较前n个字符
20
21 // 字符串查找
22 char* pos = strchr(str1, 'l');       // 查找字符第一次出现
23 char* pos2 = strrchr(str1, 'l');     // 查找字符最后一次出现
24 char* substr = strstr(str1, "world"); // 查找子串
25
26 // 内存操作 (常用于字符串)
27 memset(str1, 0, sizeof(str1));       // 清空
28 memcpy(str2, str1, len);             // 内存拷贝
29 memmove(str2, str1, len);            // 安全内存移动
30
31
32 #include <cctype>
33
34 char ch = 'A';
35
36 // 字符分类
37 isalpha(ch);    // 是否是字母
38 isdigit(ch);    // 是否是数字
39 isalnum(ch);    // 是否是字母或数字
40 isspace(ch);    // 是否是空白字符
41 isupper(ch);    // 是否是大写字母
42 islower(ch);    // 是否是字母
43 ispunct(ch);    // 是否是标点符号
44
45 // 字符转换
46 tolower(ch);    // 转小写 → 'a'
47 toupper(ch);    // 转大写 → 'A'
48
49 // 实际应用: 字符串处理
50 string s = "Hello world 123!";
51 for (char& c : s) {
52     if (isupper(c)) c = tolower(c); // 转小写
53 }
54
55
56 #include <string>
57 using namespace std;
58
59 string s = "hello world";
60
61 // 查找操作

```

```

62  size_t pos = s.find("world");           // 查找子串, 返回位置
63  pos = s.find('o');                     // 查找字符
64  pos = s.rfind('o');                     // 从后往前查找
65  pos = s.find_first_of("aeiou");         // 查找任何匹配字符
66  pos = s.find_last_of("aeiou");          // 从后往前查找任何匹配字符
67  pos = s.find_first_not_of("he lo ");    // 查找第一个不匹配字符
68
69  // 子串操作
70  string sub = s.substr(6, 5);             // 从位置6开始取5个字符 → "world"
71  s.substr(6);                             // 从位置6到结尾 → "world"
72
73  // 修改操作
74  s.insert(5, " beautiful");              // 插入 → "hello beautiful world"
75  s.erase(5, 10);                         // 删除 → "hello world"
76  s.replace(6, 5, "everyone");             // 替换 → "hello everyone"
77
78  // 追加和连接
79  s.append("!!!");                        // 追加 → "hello world!!!"
80  s += "!!!";                             // 同样效果
81  s.push_back('!');                       // 追加单个字符
82
83  // 比较
84  int result = s.compare("hello");         // 比较字符串
85  bool equal = (s == "hello world");      // 直接比较
86
87  // 数值转换
88  string num_str = to_string(123);         // 数字转字符串 → "123"
89  int num = stoi("456");                  // 字符串转int
90  double d = stod("3.14");                // 字符串转double
91  long l = stol("1000000");               // 字符串转long
92
93
94
95  #include <sstream>
96  #include <string>
97  using namespace std;
98
99  // 字符串分割
100 string s = "apple,banana,orange";
101 stringstream ss(s);
102 string token;
103 while (getline(ss, token, ',')) {
104     cout << token << endl; // 输出: apple banana orange
105 }
106
107 // 数字和字符串转换
108 stringstream ss2;
109 ss2 << "年龄: " << 25 << " 分数: " << 95.5;
110 string result = ss2.str(); // "年龄: 25 分数: 95.5"
111
112 // 从字符串提取数字
113 string data = "100 200 300";
114 stringstream ss3(data);
115 int a, b, c;
116 ss3 >> a >> b >> c; // a=100, b=200, c=300
117
118
119
120 #include <bits/stdc++.h>
121 using namespace std;
122

```

```

123 // 快速字符串处理函数
124 void string_demo() {
125     string s = "Hello world";
126
127     // 1. 删除所有空格
128     s.erase(remove(s.begin(), s.end(), ' '), s.end());
129     cout << s << endl; // "Helloworld"
130
131     // 2. 字符串转小写
132     transform(s.begin(), s.end(), s.begin(), ::tolower);
133     cout << s << endl; // "helloworld"
134
135     // 3. 字符串分割（简单版）
136     string data = "apple,banana,orange";
137     replace(data.begin(), data.end(), ',', ' ');
138     stringstream ss(data);
139     string token;
140     while (ss >> token) {
141         cout << token << endl;
142     }
143
144     // 4. 检查回文
145     string palindrome = "racecar";
146     bool is_pal = equal(palindrome.begin(), palindrome.end(),
147         palindrome.rbegin());
147     cout << (is_pal ? "是回文" : "不是回文") << endl;
148
149     // 5. 字符串反转
150     reverse(s.begin(), s.end());
151     cout << s << endl; // "dlrowolleh"
152 }
153
154
155 #include <string>
156 #include <algorithm>
157
158 // 自定义便捷函数
159 inline string to_lower(string s) {
160     transform(s.begin(), s.end(), s.begin(), ::tolower);
161     return s;
162 }
163
164 inline string to_upper(string s) {
165     transform(s.begin(), s.end(), s.begin(), ::toupper);
166     return s;
167 }
168
169 inline string trim(const string& s) {
170     auto start = s.find_first_not_of(" \t\n\r");
171     auto end = s.find_last_not_of(" \t\n\r");
172     return (start == string::npos) ? "" : s.substr(start, end - start + 1);
173 }
174
175 inline vector<string> split(const string& s, char delimiter) {
176     vector<string> tokens;
177     string token;
178     istringstream tokenStream(s);
179     while (getline(tokenStream, token, delimiter)) {
180         if (!token.empty()) tokens.push_back(token);
181     }
182     return tokens;

```

计算几何

2D

```

1 //注意所有有关vector数组都不可开过大如 n+5, 必须是有多少点n就是多少
2 //注意所有有关vector数组都不可开过大如 n+5, 必须是有多少点n就是多少
3 //注意所有有关vector数组都不可开过大如 n+5, 必须是有多少点n就是多少
4 //注意所有有关vector数组都不可开过大如 n+5, 必须是有多少点n就是多少
5 //注意所有有关vector数组都不可开过大如 n+5, 必须是有多少点n就是多少
6 //注意所有有关vector数组都不可开过大如 n+5, 必须是有多少点n就是多少
7
8 const double PI = acos(-1), eps = 1e-9;
9
10 int sgn(double x) {
11     if (fabs(x) < eps) return 0;
12     return x < 0 ? -1 : 1;
13 }
14
15 struct Pt{
16     double x, y;
17     Pt(double x = 0.0, double y = 0.0) : x(x), y(y) {}
18
19     Pt operator- (const Pt& b) const {return Pt(x - b.x, y - b.y);}
20
21     Pt operator+ (const Pt& b) const {return Pt(x + b.x, y + b.y);}
22
23     double operator% (const Pt& b) const {return x * b.y - y * b.x;}
24
25     Pt operator* (const double b) const {return Pt(x * b, y * b);}
26
27     double operator* (const Pt& b) const {return x * b.x + y * b.y;}
28
29     Pt operator/ (const double b) const {return Pt(x / b, y / b);}
30
31     bool operator< (const Pt& b) const {return x < b.x || (x == b.x && y < b.y);}
32
33     bool operator== (const Pt& b) const {return sgn(x - b.x) == 0 && sgn(y - b.y)
== 0;}
34
35     bool operator!= (const Pt& b) const {return sgn(x - b.x) != 0 || sgn(y - b.y)
!= 0;}
36 };
37
38 double dis(Pt a){
39     return sqrt(a * a);
40 }
41
42 double ddis(Pt a){
43     return a * a;
44 }
45
46 double Angle(Pt a, Pt b){
47     double val = a * b / dis(a) / dis(b);
48     return acos(max(-1.0, min(1.0, val)));

```

```

49 }
50
51 //在直线的哪边
52 // 点在直线上, 返回 0 (三点共线)
53 // 点在直线的逆时针方向, 返回 1
54 // 点在直线的顺时针方向, 返回 -1
55 //直线ab, 点c
56 int Cross(Pt a, Pt b, Pt c){
57     return sgn((b - a) % (c - a));
58 }
59
60
61 double cross(Pt a, Pt b, Pt c){
62     return (b - a) % (c - a);
63 }
64
65 struct Line {
66     Pt s, e;
67     double ang;
68     Line() {}
69     Line(Pt x, Pt y) : s(x), e(y) {
70         ang = atan2(e.y - s.y, e.x - s.x);
71     }
72
73     // 检查点 P 是否在向量 se 的右侧 (非法区域)
74     // 使用叉积判断:  $(e-s) \wedge (P-s) < 0$  则在右侧
75     bool onRight(const Pt& P) const {
76         return sgn((e - s) % (P - s)) < 0;
77     }
78
79     // 极角排序: 极角相同则保留靠左的那条
80     bool operator< (const Line& b) const {
81         if (sgn(ang - b.ang) != 0) return ang < b.ang;
82         return sgn((e - s) % (b.e - s)) > 0;
83     }
84 };
85
86
87 int Cross(Pt c, Line l){
88     Pt a = l.s, b = l.e;
89     return sgn((b - a) % (c - a));
90 }
91
92 //三点共线
93 bool In_one_line(Pt a, Pt b, Pt c){
94     return !sgn((b - a) % (c - a));
95 }
96 //点到直线距离
97 double dist_ptl(Pt p, Pt a, Pt b){
98     Pt v1 = p - a, v2 = b - a;
99     return fabs((v1 % v2) / dis(v2));
100 }
101 //点到线段距离
102 double dist_pts(Pt p, Pt a, Pt b){
103     if(a == b) return dis(p - a);
104     Pt v1 = b - a, v2 = p - a, v3 = p - b;
105     if(sgn(v1 * v2) < 0) return dis(p - a);
106     if(sgn(v1 * v3) > 0) return dis(p - b);
107     return fabs((v1 % v2) / dis(v1));
108 }
109 //点在线段上

```

```

110 bool OnSegment(Pt p, Pt a, Pt b){
111     Pt pa = a - p, pb = b - p;
112     return sgn(pa % pb) == 0 && sgn(pa * pb) <= 0;
113 }
114 //直线与线段相交(线段ab,直线cd)
115 bool Intersect_line_seg(Pt a, Pt b, Pt c, Pt d){
116     return Cross(a, b, c) * Cross(a, b, d) <= 0;
117 }
118 //线段与线段是否相交
119 bool Intersect_seg(Pt a, Pt b, Pt c, Pt d){
120     if (OnSegment(a, c, d) || OnSegment(b, c, d) || OnSegment(c, a, b) ||
OnSegment(d, a, b)) return 1;
121     if (Cross(a, b, c) * Cross(a, b, d) >= 0) return 0;
122     if (Cross(c, d, a) * Cross(c, d, b) >= 0) return 0;
123     return 1;
124 }
125 //线段到线段距离
126 double dist_sts(Pt a, Pt b, Pt c, Pt d){
127     if(Intersect_seg(a, b, c, d)){
128         return 0.0;
129     }
130     return min({
131         dist_pts(a, c, d),
132         dist_pts(b, c, d),
133         dist_pts(c, a, b),
134         dist_pts(d, a, b),
135     });
136 }
137 //直线平行
138 bool Line_parallel(Line A, Line B){
139     return sgn((A.s - A.e) % (B.s - B.e)) == 0;
140 }
141 //直线交点(四个点)
142 Pt Inter_Line_Pt(Pt a, Pt b, Pt c, Pt d){
143     Pt u = b - a, v = d - c;
144     double t = ((a - c) % v) / (v % u);
145     return a + u * t;
146 }
147
148 // 直线交点: 输入两条 Line 对象(方便半平面交调用)
149 Pt Inter_Line_Pt(Line a, Line b) {
150     Pt u = a.s - b.s;
151     Pt v1 = a.e - a.s;
152     Pt v2 = b.e - b.s;
153     double t = (v2 % u) / (v1 % v2);
154     return a.s + v1 * t;
155 }
156
157 //半平面交
158 //保留每一条直线的左侧区域
159 //!!!注意, 设置边界的时候, limit一定要比eps大一点, 不然可能卡精度, 例如limit=1e-11,eps=1e-
18
160 // L.push_back(Line(Pt(-INF, limit), Pt(-limit, limit), 0));
161 // L.push_back(Line(Pt(-limit, limit), Pt(-limit, INF), 0));
162 // L.push_back(Line(Pt(-limit, INF), Pt(-INF, INF), 0));
163 // L.push_back(Line(Pt(-INF, INF), Pt(-INF, limit), 0));
164 vector<Pt> getHalfPlaneIntersection(vector<Line>& L) {
165     // 1. 排序
166     sort(L.begin(), L.end());
167
168     // 2. 去重(对于极角相同的直线, 只保留排序后第一条, 即最靠左的那条)

```

```

169     vector<Line> ql;
170     for (int i = 0; i < L.size(); i++) {
171         if (i == 0 || sgn(L[i].ang - L[i-1].ang) != 0) {
172             ql.push_back(L[i]);
173         }
174     }
175
176     // 3. 双端队列求解
177     deque<Line> dq;
178     deque<Pt> pts; // pts[i] 是 dq[i] 和 dq[i+1] 的交点
179
180     dq.push_back(ql[0]);
181     for (int i = 1; i < ql.size(); i++) {
182         // 如果队尾的交点在当前直线的右侧（非法），弹出队尾
183         while (!pts.empty() && ql[i].onRight(pts.back())) {
184             pts.pop_back();
185             dq.pop_back();
186         }
187         // 如果队首的交点在当前直线的右侧（非法），弹出队首
188         // 这一步是为了处理“环绕一圈后覆盖了起始点”的情况
189         while (!pts.empty() && ql[i].onRight(pts.front())) {
190             pts.pop_front();
191             dq.pop_front();
192         }
193
194         dq.push_back(ql[i]);
195         // 计算新加入直线与上一条直线的交点
196         if (dq.size() > 1) {
197             pts.push_back(Inter_Line_Pt(dq[dq.size() - 2], dq.back()));
198         }
199     }
200
201     // 4. 收尾检查：队首的直线可能会切掉队尾的点
202     while (!pts.empty() && dq.front().onRight(pts.back())) {
203         pts.pop_back();
204         dq.pop_back();
205     }
206
207     // 如果队列中直线少于3条，无法构成多边形
208     if (dq.size() < 3) return {};
209
210     // 5. 计算首尾交点，封闭多边形
211     pts.push_back(Inter_Line_Pt(dq.back(), dq.front()));
212
213     // 将 deque 转为 vector 返回
214     vector<Pt> res;
215     for (auto p : pts) res.push_back(p);
216     return res;
217 }
218
219 // ----- 多边形 -----
220 // 三角形面积
221 double Triangle_area(Pt A, Pt B, Pt C){
222     return fabs((B - A) % (C - A)) / 2;
223 }
224 //凸多边形面积
225 // 因为叉积求得的三角形面积是有向的，在外面的面积可以正负抵消掉
226 // 所以能够求任意多边形面积(凸, !凸)
227 // p[]下标从 0 开始，长度为 n
228 double area(vector<Pt>& p){
229     int n = p.size();

```



```

230     double S = 0;
231     for(int i = 1; i <= n - 2; i++){
232         S += (p[i] - p[0]) % (p[i + 1] - p[0]);
233     }
234     return fabs(S / 2); // 无向面积
235     //return S / 2; // 有向面积
236 }
237
238 // 点在多边形内 (扫描线)
239 // 适用于任意多边形, 不用考虑精度误差和多边形的给出顺序
240 // 点在多边形边上, 返回 -1
241 // 点在多边形内, 返回 1
242 // 点在多边形外, 返回 0
243
244 // p[] 的下标从 0 开始, 长度为 n
245 int InPolygon(Pt P, vector<Pt>& p) {
246     int n = p.size();
247     bool flag = false;
248     for (int i = 0, j = n - 1; i < n; j = i++) {
249         Pt p1 = p[i], p2 = p[j];
250         if (OnSegment(P, p1, p2)) return -1;
251         if (sgn(P.y - p1.y) > 0 == sgn(P.y - p2.y) > 0) continue;
252         if (sgn((P.y - p1.y) * (p1.x - p2.x) / (p1.y - p2.y) + p1.x - P.x) > 0)
253             flag = !flag;
254     }
255     return flag;
256 }
257 // 是否为凸多边形
258 bool Is_convex(vector<Pt>& p){
259     int n = p.size();
260     bool s[3] = {0, 0, 0};
261     for (int i = 0, j = n - 1, k = n - 2; i < n; k = j, j = i++) {
262         int cnt = sgn((p[i] - p[j]) % (p[k] - p[j])) + 1;
263         s[cnt] = true;
264         if (s[0] && s[2]) return false;
265     }
266     return true;
267 }
268
269
270 // ----- 圆 -----
271 struct Circle{
272     Pt o;
273     double r;
274     Circle(Pt o = Pt(), double r = 0) : o(o), r(r) {}
275
276     double S(){return PI * r * r;}
277     double C(){return PI * 2 * r;}
278 };
279 // 扇形面积
280 double SectorArea(Pt A, Pt B, double R){
281     double angle = Angle(A, B);
282     if(sgn(A % B) < 0) angle = -angle;
283     return R * R * angle / 2;
284 }
285 // 点和圆的位置关系
286 // 点在圆上, 返回 0
287 // 点在圆外, 返回 -1
288 // 点在圆内, 返回 1
289 int PWC(Pt p, Circle c){
290     double d = dis(p - c.o);

```

```

291     if(sgn(d - c.r) == 0) return 0;
292     if(sgn(d - c.r) > 0) return -1;
293     return 1;
294 }
295 //直线和圆的位置关系
296 // 相切, 返回 0
297 // 相交, 返回 1
298 // 相离, 返回 -1
299 int LWC(Pt A, Pt B, Circle c) {
300     double d = dist_ptl(c.o, A, B);
301     if (sgn(d - c.r) == 0) return 0;
302     if (sgn(d - c.r) > 0) return -1;
303     return 1;
304 }
305
306 //直线和圆的交点
307 vector<Pt> Intersection_line_circle(Pt A, Pt B, Circle c) {
308     Pt AB = B - A;
309     double len2 = AB * AB;
310     Pt pr = A + AB * ((c.o - A) * AB / len2);
311     double d2 = ddis(pr - c.o);
312
313     int status = sgn(d2 - c.r * c.r);
314     if (status > 0) return {}; // 相离, 返回空 vector
315
316     double base = sqrt(max(0.0, c.r * c.r - d2));
317     if (status == 0) return {pr}; // 相切, 返回 1 个点
318
319     // 相交, 返回 2 个点
320     Pt e = AB / sqrt(len2);
321     return {pr + e * base, pr - e * base};
322 }
323 //圆与圆的位置关系
324 // 相离, 返回 -1
325 // 外切, 返回 0
326 // 内切(A 包含 B), 返回 1
327 // 内切(B 包含 A), 返回 2
328 // 内含(A 包含 B), 返回 3
329 // 内含(B 包含 A), 返回 4
330 // 相交, 返回 5
331
332 int Circle_with_circle(Circle A, Circle B) {
333     double len1 = dis(A.o - B.o);
334     double len2 = A.r + B.r;
335     if (sgn(len1 - len2) > 0) return -1;
336     if (sgn(len1 - len2) == 0) return 0;
337     if (sgn(len1 + len2 - 2 * A.r) == 0) return 1;
338     if (sgn(len1 + len2 - 2 * B.r) == 0) return 2;
339     if (sgn(len1 + len2 - 2 * A.r) < 0) return 3;
340     if (sgn(len1 + len2 - 2 * B.r) < 0) return 4;
341     return 5;
342 }
343
344 //圆与圆的交点
345 // 相交, 返回两点坐标
346 // 相切, 返回两个一样的相切点
347
348 // 要先判断是否相交或相切再调用
349 vector<Pt> Intersection_circle_circle(Circle A, Circle B) {
350     double d = dis(A.o - B.o);
351     // 情况 1: 相离或内含 (无交点)

```

```

352     if (sgn(d - (A.r + B.r)) > 0 || sgn(d - fabs(A.r - B.r)) < 0) {
353         return {};
354     }
355
356     // 情况 2: 相切 (1 个交点)
357     if (sgn(d - (A.r + B.r)) == 0 || sgn(d - fabs(A.r - B.r)) == 0) {
358         Pt e = (B.o - A.o) / d;
359         return {A.o + e * A.r};
360     }
361
362     // 情况 3: 相交 (2 个交点)
363     double a = acos(max(-1.0, min(1.0, (A.r * A.r + d * d - B.r * B.r) / (2.0 *
A.r * d)))));
364     double t = atan2(B.o.y - A.o.y, B.o.x - A.o.x);
365     return {
366         A.o + Pt(A.r * cos(t + a), A.r * sin(t + a)),
367         A.o + Pt(A.r * cos(t - a), A.r * sin(t - a))
368     };
369 }
370 //求圆外一点对圆的两个切点
371 vector<Pt> TangentPt_Pt_circle(Pt p, Circle c) {
372     double d2 = ddis(p - c.o);
373     int status = sgn(d2 - c.r * c.r);
374
375     if (status < 0) return {}; // 点在圆内, 无切点
376     if (status == 0) return {p}; // 点在圆上, 切点就是自身
377
378     // 点在圆外, 2 个切点
379     double d = sqrt(d2);
380     double l = sqrt(d2 - c.r * c.r);
381     double angle = asin(max(-1.0, min(1.0, c.r / d)));
382     Pt e = (c.o - p) / d;
383
384     // 旋转向量得到切点方向
385     auto rotate = [](Pt v, double ang) {
386         return Pt(v.x * cos(ang) - v.y * sin(ang), v.x * sin(ang) + v.y *
cos(ang));
387     };
388
389     return {
390         p + rotate(e, angle) * l,
391         p + rotate(e, -angle) * l
392     };
393 }
394
395 //求三角形外接圆
396 Circle get_circumcircle(Pt A, Pt B, Pt C) {
397     double Bx = B.x - A.x, By = B.y - A.y;
398     double Cx = C.x - A.x, Cy = C.y - A.y;
399     double D = 2 * (Bx * Cy - By * Cx);
400
401     double x = (Cy * (Bx * Bx + By * By) - By * (Cx * Cx + Cy * Cy)) / D + A.x;
402     double y = (Bx * (Cx * Cx + Cy * Cy) - Cx * (Bx * Bx + By * By)) / D + A.y;
403     Pt P(x, y);
404     return Circle(P, dis(A - P));
405 }
406 //三角形内切圆
407 Circle get_incircle(Pt A, Pt B, Pt C) {
408     double a = dis(B - C);
409     double b = dis(A - C);
410     double c = dis(A - B);

```

```

411     Pt p = (A * a + B * b + C * c) / (a + b + c);
412     return Circle(p, dist_pt1(p, A, B));
413 }
414 // 要保证传入的点是整点
415 // 线段上的整点个数
416 // 注意按照要求修改返回值
417 int IntegerPt_on_seg(Pt A, Pt B) {
418     int x = abs(A.x - B.x);
419     int y = abs(A.y - B.y);
420     if (x == 0 || y == 0) return 1;
421     return __gcd(x, y) + 1; // 包含端点
422     return __gcd(x, y) - 1; // 不包含端点
423 }
424 // 返回多边形边**上**整点的个数
425 // 点需要是顺时针(逆时针)给出
426
427 // p[] 下标从 0 开始, 长度为 n
428 int IntegerPt_on_polygon(vector<Pt>& p) {
429     int n = p.size();
430     int res = 0;
431     for (int i = 0, j = n - 1; i < n; j = i++) {
432         int x = abs(p[i].x - p[j].x);
433         int y = abs(p[i].y - p[j].y);
434         res += __gcd(x, y);
435     }
436     return res;
437 }
438 // 返回不包括边界的, 多边形**内**整点个数
439 int IntegerPt_in_polygon(vector<Pt>& p) {
440     int n = p.size();
441     double A = area(p);
442     double B = IntegerPt_on_polygon(p);
443     return A - B / 2 + 1;
444 }
445
446 int getQuadrant(Pt p, Pt center) {
447     double x = p.x - center.x;
448     double y = p.y - center.y;
449     if (x > 0 && y >= 0) return 1; // 第一象限
450     if (x <= 0 && y > 0) return 2; // 第二象限
451     if (x < 0 && y <= 0) return 3; // 第三象限
452     if (x >= 0 && y < 0) return 4; // 第四象限
453     return 0;
454 }
455
456 void polarSort(vector<Pt>& Pts, const Pt& center) {
457     sort(Pts.begin(), Pts.end(), [&center](const Pt& a, const Pt& b) {
458         int quadA = getQuadrant(a, center);
459         int quadB = getQuadrant(b, center);
460
461         if (quadA != quadB) {
462             return quadA < quadB;
463         }
464
465         // 同一象限内使用叉积
466         double cp = (a - center) % (b - center);
467         if (fabs(cp) > eps) {
468             return cp > 0; // 逆时针排序
469         }
470
471         // 共线时, 距离近的在前

```

```

472         return ddis(a - center) < ddis(b - center);
473     });
474 }
475
476 //凸包算法
477 vector<Pt> Andrew(vector<Pt>& p){
478     sort(p.begin(), p.end());
479     int n = p.size();
480     vector<Pt> ans;
481     for(int i = 0; i < n; i++){
482         while(ans.size() >= 2 && Cross(ans[ans.size() - 2], ans.back(), p[i]) <=
0){
483             ans.pop_back();
484         }
485         ans.push_back(p[i]);
486     }
487
488     int t = ans.size();
489     for(int i = n - 2; i >= 0; i--){
490         while(ans.size() > t && Cross(ans[ans.size() - 2], ans.back(), p[i]) <=
0){
491             ans.pop_back();
492         }
493         ans.push_back(p[i]);
494     }
495
496     if(ans.size() > 1) ans.pop_back();
497     return ans;
498 }
499
500
501 int norm_idx(int x, int n){
502     x %= n;
503     if(x < 0) x += n;
504     return x;
505 }
506
507 // 点在已排序凸包中的位置:
508 // 返回 -1 表示外部, 0 表示边界上, 1 表示内部
509 // 要求 hull 为逆时针排序的凸包
510 int InConvex(Pt q, const vector<Pt>& hull){
511     int n = hull.size();
512     if(n == 0) return -1;
513     if(n == 1) return q == hull[0] ? 0 : -1;
514     if(n == 2) return OnSegment(q, hull[0], hull[1]) ? 0 : -1;
515
516     int s1 = Cross(hull[0], hull[1], q);
517     int s2 = Cross(hull[0], hull[n - 1], q);
518     if(s1 < 0 || s2 > 0) return -1;
519     if(OnSegment(q, hull[0], hull[1]) || OnSegment(q, hull[0], hull[n - 1]))
return 0;
520
521     int l = 1, r = n - 1;
522     while(l + 1 < r){
523         int mid = (l + r) >> 1;
524         if(Cross(hull[0], hull[mid], q) >= 0) l = mid;
525         else r = mid;
526     }
527
528     int side = Cross(hull[1], hull[(l + 1) % n], q);
529     if(side < 0) return -1;

```

```

530     if(OnSegment(q, hull[l], hull[(l + 1) % n])) return 0;
531     return side == 0 ? 0 : 1;
532 }
533
534 // 若 q 在凸包外部, 返回一个可见边的编号 i, 表示边 hull[i] -> hull[i+1] 对 q 可见
535 // 若 q 在凸包内部或边界上, 返回 -1
536 int FindVisibleEdge(Pt q, const vector<Pt>& hull){
537     int n = hull.size();
538     if(n < 3) return -1;
539
540     if(Cross(hull[0], hull[1], q) < 0) return 0;
541     if(Cross(hull[0], hull[n - 1], q) > 0) return n - 1;
542
543     int l = 1, r = n - 1;
544     while(l + 1 < r){
545         int mid = (l + r) >> 1;
546         if(Cross(hull[0], hull[mid], q) >= 0) l = mid;
547         else r = mid;
548     }
549
550     return Cross(hull[l], hull[(l + 1) % n], q) < 0 ? l : -1;
551 }
552
553 bool VisibleEdge(Pt q, const vector<Pt>& hull, int idx){
554     int n = hull.size();
555     return Cross(hull[norm_idx(idx, n)], hull[norm_idx(idx + 1, n)], q) < 0;
556 }
557
558 // 返回外点 q 到已排序凸包的两个切点下标
559 // 返回值中的 first, second 分别是可见边链的起点和终点, 按凸包逆时针顺序给出
560 // 若 q 不在凸包外部, 则返回 {-1, -1}
561 pair<int, int> TangentIndexConvex(Pt q, const vector<Pt>& hull){
562     int n = hull.size();
563     if(n == 0) return {-1, -1};
564     if(n == 1) return {0, 0};
565     if(n == 2) return {0, 1};
566
567     int k = FindVisibleEdge(q, hull);
568     if(k == -1) return {-1, -1};
569
570     int lo = 0, hi = n - 1;
571     while(lo < hi){
572         int mid = (lo + hi + 1) >> 1;
573         if(VisibleEdge(q, hull, k + mid)) lo = mid;
574         else hi = mid - 1;
575     }
576     int right_len = lo;
577
578     lo = 0, hi = n - 1;
579     while(lo < hi){
580         int mid = (lo + hi + 1) >> 1;
581         if(VisibleEdge(q, hull, k - mid)) lo = mid;
582         else hi = mid - 1;
583     }
584     int left_len = lo;
585
586     int left_edge = norm_idx(k - left_len, n);
587     int right_edge = norm_idx(k + right_len, n);
588     return {left_edge, norm_idx(right_edge + 1, n)};
589 }
590

```

```

591 pair<Pt, Pt> TangentPointConvex(Pt q, const vector<Pt>& hull){
592     auto idx = TangentIndexConvex(q, hull);
593     if(idx.first == -1) return {Pt(), Pt()};
594     return {hull[idx.first], hull[idx.second]};
595 }
596
597 bool InTangentCone(Pt a, Pt b, const vector<Pt>& hull){
598     auto idx = TangentIndexConvex(a, hull);
599     if(idx.first == -1) return InConvex(a, hull) >= 0;
600     return Cross(a, hull[idx.first], b) <= 0 && Cross(a, hull[idx.second], b) >=
0;
601 }
602
603 // 判断线段 ab 是否与已排序凸包相交
604 // 要求 hull 为逆时针排序的凸包
605 bool Intersect_seg_convex(Pt a, Pt b, const vector<Pt>& hull){
606     int n = hull.size();
607     if(n == 0) return false;
608     if(n == 1) return OnSegment(hull[0], a, b);
609     if(n == 2) return Intersect_seg(a, b, hull[0], hull[1]);
610
611     if(InConvex(a, hull) >= 0 || InConvex(b, hull) >= 0) return true;
612     return InTangentCone(a, b, hull) && InTangentCone(b, a, hull);
613 }
614

```

3D

```

1  const double PI = acos(-1);
2
3  int sgn(double x) {
4      if (fabs(x) < 1e-8) return 0;
5      return x < 0 ? -1 : 1;
6  }
7
8  struct Point{
9      double x, y, z;
10     Point(double x = 0, double y = 0, double z = 0) : x(x), y(y), z(z) {}
11
12     Point operator+ (const Point& B) const {return Point(x + B.x, y + B.y, z +
B.z);}
13
14     Point operator- (const Point& B) const {return Point(x - B.x, y - B.y, z -
B.z);}
15     //点乘
16     double operator* (const Point& B) const {return x * B.x + y * B.y + z * B.z;}
17     //叉乘
18     Point operator% (const Point& B) const {return Point(y * B.z - z * B.y, z *
B.x - x * B.z, x * B.y - y * B.x);}
19     //数乘
20     Point operator* (const double& B) const {return Point(x * B, y * B, z * B);}
21     //数除 (
22     Point operator/ (const double& B) const {return Point(x / B, y / B, z / B);}
23
24     bool operator== (const Point& B) const {return sgn(x - B.x) == 0 && sgn(y -
B.y) == 0 && sgn(z - B.z) == 0;}
25

```

```

26     bool operator!= (const Point& B) const {return sgn(x - B.x) != 0 || sgn(y -
    B.y) != 0 || sgn(z - B.z) != 0;}
27
28
29 };
30
31 double dis(Point& a){
32     return sqrt(a * a);
33 }
34
35 double ddis(Point& a){
36     return a * a;
37 }
38
39 // 两点间距离
40 double dis(Point& a, Point& b){
41     return sqrt((a - b) * (a - b));
42 }
43 //两点间距离平方
44 double ddis(Point& a, Point& b){
45     return (a - b) * (a - b);
46 }
47 //向量夹角
48 double Angle(Point& a, Point& b){
49     return acos(a * b / dis(a) / dis(b));
50 }

```

动态规划

斜率优化DP

```

1  int mod=1e9+7;
2  int minProblem=1;
3  //由于dp的形式千变万化，这里仅供思路参考
4  struct pt{
5      int x,y;
6  };
7
8  //点p1,p2的斜率是否小于k
9  bool check(pt p1,pt p2,int k){
10     int c=(p2.y-p1.y);
11     int d=k*(p2.x-p1.x);
12     if(minProblem)return (p2.y-p1.y)<=k*(p2.x-p1.x);
13     else return (p2.y-p1.y)>=k*(p2.x-p1.x);
14 }
15
16 //点p1,p2的斜率是否大于p2,p3
17 bool check(pt p1,pt p2,pt p3){
18     if(minProblem)return (p2.y-p1.y)*(p3.x-p2.x)>=(p3.y-p2.y)*(p2.x-p1.x);
19     else return (p2.y-p1.y)*(p3.x-p2.x)<=(p3.y-p2.y)*(p2.x-p1.x);
20 }
21
22
23 void solve3(){
24     int n;
25     cin>>n;
26     vector<int> s(n+1);

```



```

27     vector<int> k(n+1);
28     vector<int> c(n+1);
29     vector<int> kk(n+1);
30     vector<int> ss(n+1);
31     for(int i=1;i<=n;i++){
32         cin>>s[i]>>k[i]>>c[i];
33         kk[i]=k[i]+kk[i-1];
34         ss[i]=k[i]*s[i]+ss[i-1];
35     }
36
37
38     //dp[i]=min{dp[j]+ss[j]-ss[i]+s[i]*kk[i]-s[i]*kk[j]+c[i]}
39     //dp[i]+ss[i]-s[i]*kk[i]-c[i]=dp[j]+ss[j]-s[i]*kk[j];
40     //因为s[i]已知且单调不减, 使用斜率优化DP
41     //设b=y-k*x    b=dp[i]+ss[i]-s[i]*kk[i]-c[i]    y=dp[j]+ss[j]    k=s[i]    x=kk[j]
42
43     //如果维护max问题, 则维护上凸包, k取负数, minproblem=0
44     //如果维护min问题, 则维护下凸包, k取正数, minproblem=1
45
46     // (注意负号! 我们保证k是正数, 所以x可能为了拼凑负号变为负数)
47     //得到的新点{kk[i], dp[i]+ss[i]}插入单调队列中, 保证单调队列中两点间斜率单调递增
48     //使用一个cur维护当前符合要求的点 (要么是终点, 要么cur与cur+1两点间斜率刚好是单调队列中比k稍
    大的点)
49     //时间复杂度O(n)
50     //以下部分可以作为模板参考, 替换其中的x,y,k,b的计算方式, 只要保证k单调就可以应用
51     vector<pt> dp;    //dp单调队列
52     vector<int> adp(n+1);    //dp答案
53     dp.push_back({0,0});
54     int cur=0;
55     for(int i=1;i<=n;i++){
56         int k=s[i];
57         while(cur+1<dp.size()&&!check(dp[cur], dp[cur+1], k)){
58             cur++;
59         }
60
61         int x=dp[cur].x;
62         int y=dp[cur].y;
63         int b=y-x*k;
64         adp[i]=b-ss[i]+s[i]*kk[i]+c[i];
65
66         //将转移方程中x,y项中的j替换为i即可
67         int nx=kk[i];
68         int ny=adp[i]+ss[i];
69
70         while(dp.size()-2>=0&&check(dp[dp.size()-2], dp[dp.size()-1], {nx,ny})){
71             dp.pop_back();
72         }
73
74         dp.push_back({nx,ny});
75     }
76     int ans=adp[n];
77     cur=n;
78
79     //此题特判最后k为0无贡献
80     while(cur&&k[cur]==0){
81         cur--;
82         ans=min(ans, adp[cur]);
83     }
84     cout<<ans<<"\n";
85
86 }

```

杂项

高精度加减乘除

```
1 string removeLeadingZeros(string s){
2     int pos = s.find_first_not_of('0');
3     if(pos == string::npos) return "0";
4     return s.substr(pos);
5 }
6
7 string addBigNumbers(string num1, string num2){
8     int maxLength = max(num1.size(), num2.size());
9
10    while(num1.size() < maxLength) num1 = "0" + num1;
11    while(num2.size() < maxLength) num2 = "0" + num2;
12
13    string result = "";
14
15    int carry = 0;
16
17    for(int i = maxLength - 1; i >= 0; i--){
18        int digit1 = num1[i] - '0';
19        int digit2 = num2[i] - '0';
20
21        int sum = digit1 + digit2 + carry;
22
23        result.push_back(sum % 10 + '0');
24
25        carry = sum / 10;
26    }
27
28    if(carry){
29        result.push_back(carry + '0');
30    }
31
32    reverse(result.begin(), result.end());
33
34    return removeLeadingZeros(result);
35 }
36
37 string subtractBigNumbers(string num1, string num2){
38     bool isNegative = false;
39
40     if(num1.size() < num2.size() || (num1.size() == num2.size() && num1 < num2)){
41         swap(num1, num2);
42         isNegative = true;
43     }
44
45     int maxLength = max(num1.size(), num2.size());
46
47     while(num1.size() < maxLength) num1 = "0" + num1;
48     while(num2.size() < maxLength) num2 = "0" + num2;
49
50     string result = "";
51
52     int borrow = 0;
```

```

53
54     for(int i = maxLength - 1; i >= 0; i--){
55         int digit1 = num1[i] - '0';
56         int digit2 = num2[i] - '0';
57
58         digit1 -= borrow;
59
60         borrow = 0;
61
62         if(digit1 < digit2){
63             digit1 += 10;
64             borrow = 1;
65         }
66
67         int diff = digit1 - digit2;
68
69         result.push_back(diff + '0');
70     }
71
72     reverse(result.begin(), result.end());
73
74     result = removeLeadingZeros(result);
75
76     if(isNegative && result != "0"){
77         result = "-" + result;
78     }
79
80     return result;
81 }
82
83 string multiplyBigNumbers(string num1, string num2){
84     if(num1 == "0" || num2 == "0") return "0";
85
86     int n = num1.size();
87     int m = num2.size();
88
89     string result(n + m, '0');
90
91     for(int i = n - 1; i >= 0; i--){
92         for(int j = m - 1; j >= 0; j--){
93             int mul = (num1[i] - '0') * (num2[j] - '0');
94
95             int sum = (result[i + j + 1] - '0') + mul;
96
97             result[i + j + 1] = sum % 10 + '0';
98
99             result[i + j] += sum / 10;
100         }
101     }
102
103     return removeLeadingZeros(result);
104 }
105
106 string divideBigNumbers(string num1, int divisor){
107     string result = "";
108
109     long long cur = 0;
110
111     for(int i = 0; i < num1.size(); i++){
112         cur = cur * 10 + (num1[i] - '0');
113

```

```

114         result.push_back(cur / divisor + '0');
115
116         cur %= divisor;
117     }
118
119     return removeLeadingZeros(result);
120 }
121
122 int main(){
123     string a, b;
124
125     cin >> a >> b;
126
127     cout << addBigNumbers(a, b) << '\n';
128
129     cout << subtractBigNumbers(a, b) << '\n';
130
131     cout << multiplyBigNumbers(a, b) << '\n';
132
133     cout << divideBigNumbers(a, stoi(b)) << '\n';
134
135     return 0;
136 }

```

对拍

bf.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  //暴力内容
5  int main(){
6      int n, m;
7      cin >> n >> m;
8      int ans = 0;
9      while(n--) ans++;
10     while(m--) ans++;
11     cout << ans;
12 }

```

solve.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  //程序正解内容
5  int main(){
6      int n, m;
7      cin >> n >> m;
8      cout << n + m;
9  }

```

data.cpp

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4
5  mt19937_64 rd(random_device{}());
6
7  //生成随机数据作为输入
8  int rnd(int l, int r){
9      return l + rd() % (r - l + 1);
10 }
11
12 signed main(){
13     cout << rnd(1, 100) << " " << rnd(1, 100);
14     return 0;
15 }
```

test.cpp

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4
5  //链接以上程序
6  signed main(){
7      int t = 1000;
8      for(int i = 1; i <= t; i++){
9
10         /*
11         system("./data > data.in");
12         system("./solve < data.in > solve.out");
13         system("./bf < data.in > bf.out");
14         */
15
16         system("data.exe > data.in");
17         system("solve.exe < data.in > solve.out");
18         system("bf.exe < data.in > bf.out");
19
20         /*
21         if(system("diff solve.out bf.out > /dev/null"))
22             */
23         if(system("fc solve.out bf.out > nul")){
24             cout << "WA";
25             break;
26         }
27
28         cout << i << " AC\n";
29
30     }
31     return 0;
32 }
```

dpL.sh

```

1 g++ data.cpp -o data
2 g++ solve.cpp -o solve
3 g++ bf.cpp -o bf
4 g++ test.cpp -o test
5 ./test

```

dpW.bat

```

1 g++ data.cpp -o data.exe
2 g++ solve.cpp -o solve.exe
3 g++ bf.cpp -o bf.exe
4 g++ test.cpp -o test.exe
5 test.exe

```

STL库函数

```

1 #include <numeric>
2 #include <algorithm>
3 #include <cmath>
4
5 // 最大公约数
6 __gcd(a, b);           // GNU扩展, 最快
7 gcd(a, b);             // C++17标准
8
9 // 最小公倍数
10 lcm(a, b);             // C++17
11
12 // 累积计算
13 accumulate(v.begin(), v.end(), 0);           // 求和
14 accumulate(v.begin(), v.end(), 1, multiplies<int>()); // 求积
15 partial_sum(v.begin(), v.end(), result.begin()); // 前缀和
16 adjacent_difference(v.begin(), v.end(), result.begin()); // 差分
17
18 // 其他数学
19 __builtin_popcount(x); // 二进制中1的个数
20 abs(x); pow(x, y); sqrt(x); // 绝对值、幂、平方根
21
22
23 #include <algorithm>
24
25 // 二分查找（要求有序）
26 lower_bound(v.begin(), v.end(), x); // 第一个≥x的位置
27 upper_bound(v.begin(), v.end(), x); // 第一个>x的位置
28 binary_search(v.begin(), v.end(), x); // 是否存在x
29 equal_range(v.begin(), v.end(), x); // 返回[x, x]的区间
30
31 // 统计
32 count(v.begin(), v.end(), x); // 统计x出现次数
33 count_if(v.begin(), v.end(), pred); // 统计满足条件的元素
34
35 // 最值
36 max_element(v.begin(), v.end()); // 最大元素位置
37 min_element(v.begin(), v.end()); // 最小元素位置
38
39

```

```

40 // 填充
41 fill(v.begin(), v.end(), value); // 填充值
42 iota(v.begin(), v.end(), start); // 填充递增序列
43
44 // 变换
45 transform(v.begin(), v.end(), result.begin(), func); // 对每个元素应用函数
46
47 // 交换
48 swap(a, b); // 交换两个值
49 iter_swap(it1, it2); // 交换迭代器指向的值
50
51 // 反转
52 reverse(v.begin(), v.end()); // 反转序列
53
54
55
56 // 下一个排列（字典序）
57 next_permutation(v.begin(), v.end()); // 变为下一个排列
58 prev_permutation(v.begin(), v.end()); // 变为上一个排列
59
60 // 使用示例：生成所有排列
61 sort(v.begin(), v.end());
62 do {
63     // 处理当前排列
64 } while (next_permutation(v.begin(), v.end()));
65
66
67
68 // 集合运算（要求有序）
69 set_union(a.begin(), a.end(), b.begin(), b.end(), result.begin()); // 并集
70 set_intersection(a.begin(), a.end(), b.begin(), b.end(), result.begin()); // 交集
71 set_difference(a.begin(), a.end(), b.begin(), b.end(), result.begin()); // 差集
72 includes(a.begin(), a.end(), b.begin(), b.end()); // 包含关系
73
74
75
76 #include <bitset>
77 #include <bit> // C++20
78
79 bitset<32> bs(x); // 位集操作
80 __builtin_clz(x); // 前导0个数
81 __builtin_ctz(x); // 后缀0个数
82 __builtin_ffs(x); // 最低位1的位置
83 popcount(x); // C++20, 1的个数
84
85
86
87 ios::sync_with_stdio(false);
88 cin.tie(0);
89 cout.tie(0);

```

迭代器操作

```

1 #include <iterator>
2
3 // 前进n步（不改变原迭代器）
4 auto it2 = next(it, n); // 返回it后面第n个位置的迭代器
5 auto it2 = next(it); // 相当于next(it, 1)

```

```

6
7 // 后退n步（不改变原迭代器）
8 auto it2 = prev(it, n); // 返回it前面第n个位置的迭代器
9 auto it2 = prev(it); // 相当于prev(it, 1)
10
11 // 前进n步（改变原迭代器）
12 advance(it, n); // 将it前进n步
13
14 #include <iterator>
15
16 // 计算两个迭代器之间的距离
17 int dist = distance(it1, it2);
18
19 // 获取迭代器在容器中的位置索引
20 int pos = distance(container.begin(), it);
21
22 int main() {
23     vector<int> v = {1, 2, 3, 4, 5};
24
25     // 正向迭代器转反向迭代器
26     auto it = v.begin() + 2; // 指向3
27     auto rit = reverse_iterator<vector<int>::iterator>(it);
28     cout << *rit << endl; // 输出：2（指向前一个元素）
29
30     // 反向迭代器转正向迭代器
31     auto rit2 = v.rbegin() + 1; // 指向4
32     auto it2 = rit2.base(); // 指向5
33     cout << *it2 << endl; // 输出：5
34
35     return 0;
36 }
37

```